

SPRING 2025

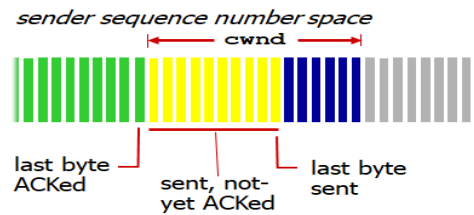
NETWORKS

NOTE 11

TCP #3

TCP Congestion control

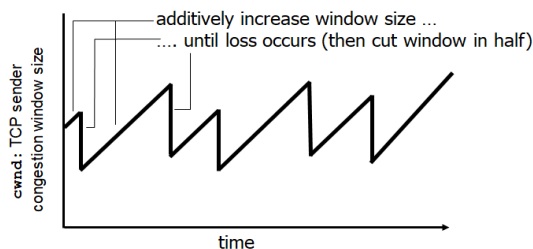
- Uses **end-to-end** congestion control



- The sender should **limits** transmission by
Last byte sent – last byte acked $\leq \text{Min} \{ \text{cwnd}, \text{rwnd} \}$
- Cwnd value is not static, it depends on the network congestion
- Rate $\approx \text{cwnd} / \text{RTT}$ “Send cwnd then wait RTT”

Congestion control approach “Probing for bandwidth”

- **additive increase**: increase **cwnd** by 1 MSS every RTT until loss detected
- **multiplicative decrease**: cut **cwnd** in half after loss



- TCP will take the arrival of the acknowledgments as an indication that all is well will **increase** its congestion window size (**self-clocking**).
- TCP congestion control has three major components
 - Slow Start (Mandatory)
 - Congestion Avoidance (Mandatory)
 - Fast Recovery (Recommended)

Slow Start

- The value of ***cwnd*** begins at 1 MSS and increases by 1 MSS every time a transmitted segment is first acknowledged.
- This process results in a doubling of the sending rate every ***RTT***.
- The TCP send rate starts slow but grows **exponentially** during the slow start phase.

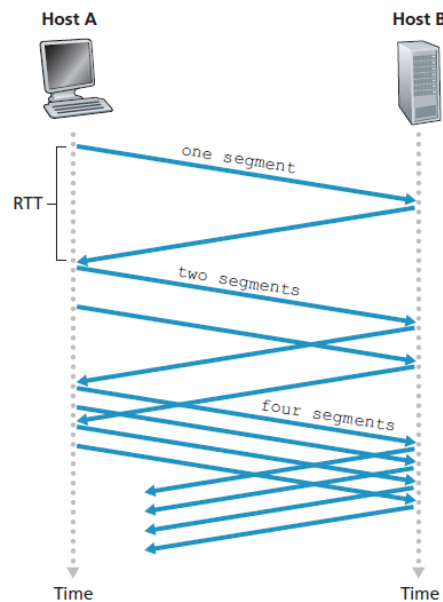


Figure 3.50 ♦ TCP slow start

End of slow start

1. If there is a **loss event** indicated by timeout, the TCP sender sets the value of ***cwnd*** to 1 and begins the slow start process anew. It also sets the value of a second state variable, ***ssthresh*** = ***cwnd***/2
2. When the value of ***cwnd*** equals ***ssthresh***, slow start ends and TCP transitions into **congestion avoidance** mode.

3. If three duplicate ACKs are detected, in which case TCP performs a **fast retransmit** and enters the **fast recovery** state.

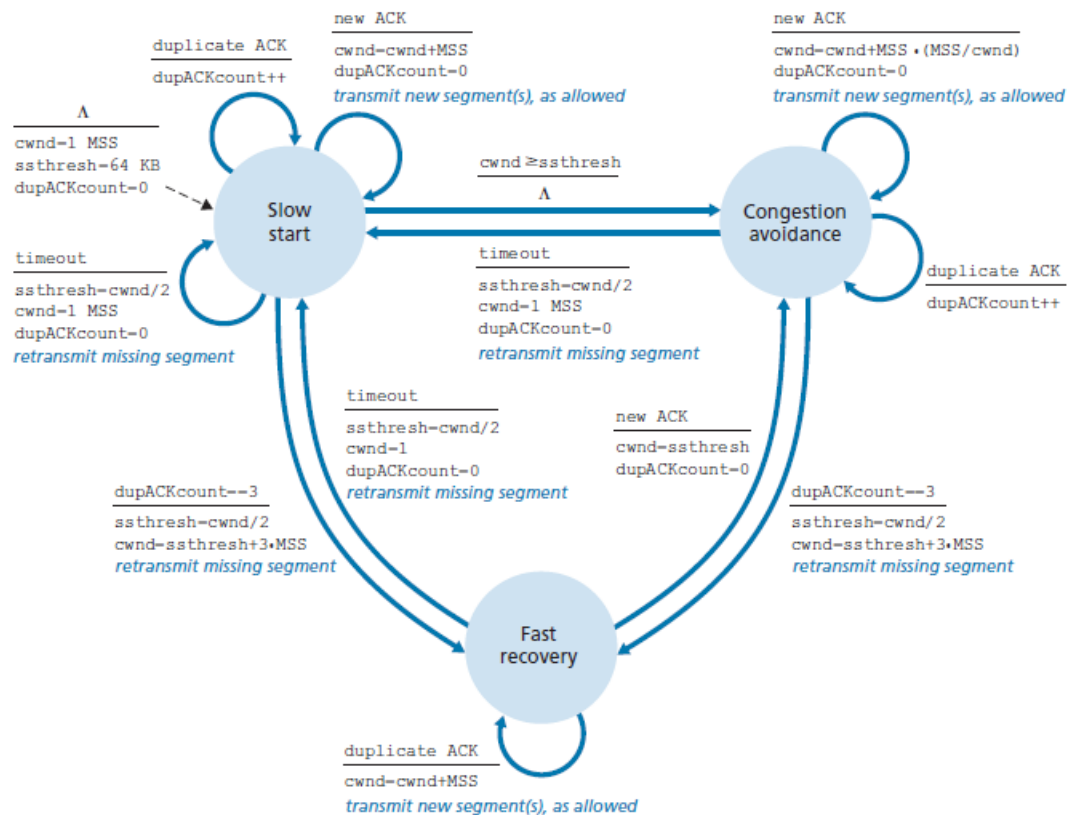


Figure 3.51 ♦ FSM description of TCP congestion control

Congestion Avoidance

- On entry to the congestion-avoidance state, the value of ***cwnd*** is approximately half its value when congestion was last encountered.
- Rather than doubling the value of ***cwnd*** every ***RTT***, increase the value of ***cwnd*** by just a single MSS every RTT.
- For example, If MSS is **1,460 bytes** and ***cwnd*** is **14,600 bytes**, then
 - 10 segments are being sent within an RTT

- Each arriving ACK increases *cwnd* by **1/10 MSS**
- the value of *cwnd* will have increased by one MSS after ACKs when all 10 segments have been received

End of congestion avoidance

- When a **timeout** occurs, the value of *cwnd* is set to 1 MSS, and the value of *ssthresh* is updated to half the value of *cwnd*. The slow start- state is then entered.
- To account for the **triple duplicate ACKs** received, TCP halves the value of *cwnd* adding in **3 MSS** and records the value of *ssthresh* to be half the value of *cwnd*. The fast-recovery state is then entered.

Fast Recovery

- Fast recovery is a **recommended**, but not a required, component of TCP.
- An early version of TCP, known as **TCP Tahoe**, unconditionally cut its congestion window to 1 MSS and entered the slow-start phase after either a timeout-indicated or triple-duplicate-ACK-indicated loss event.
- The newer version of TCP, **TCP Reno**, incorporated fast recovery.

What is the reaction on loss?

- For timeout
 - Cwnd set to **1 MSS**
 - Window then grow **exponentially** (Slow Start) to **threshold**, then grows **linearly**
- For 3 duplicate acks
 - **TCP Tahoe** → Always set MSS to 1
 - **TCP RENO** → cwnd is cut to half + 3 then grows linearly

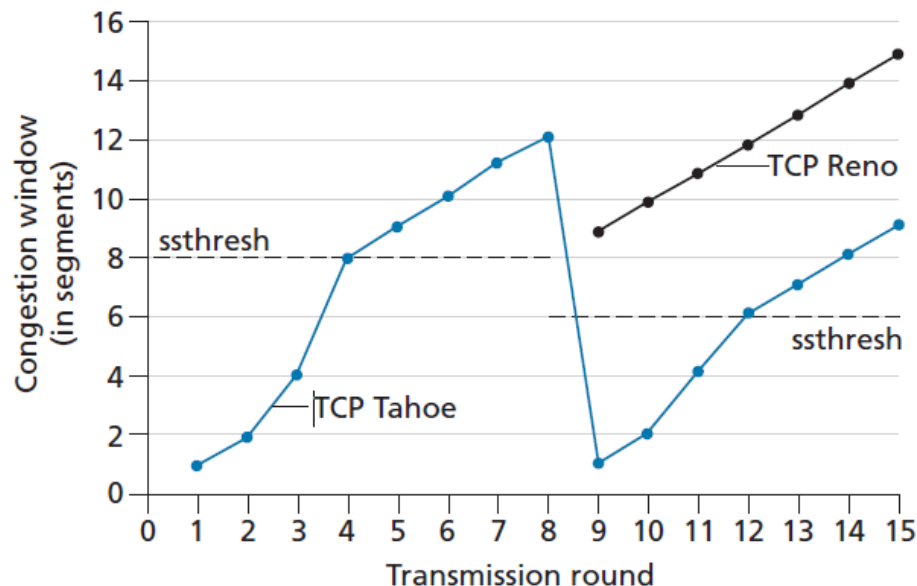


Figure 3.52 ♦ Evolution of TCP's congestion window (Tahoe and Reno)

TCP Qubic

- TCP CUBIC differs only slightly from TCP Reno.
- The congestion window is increased only on ACK receipt, and the slow start and fast recovery phases remain the same.

- CUBIC only changes the **congestion avoidance** phase and attempts to maintain the flow for as long as possible just below the (unknown to the sender) congestion threshold.

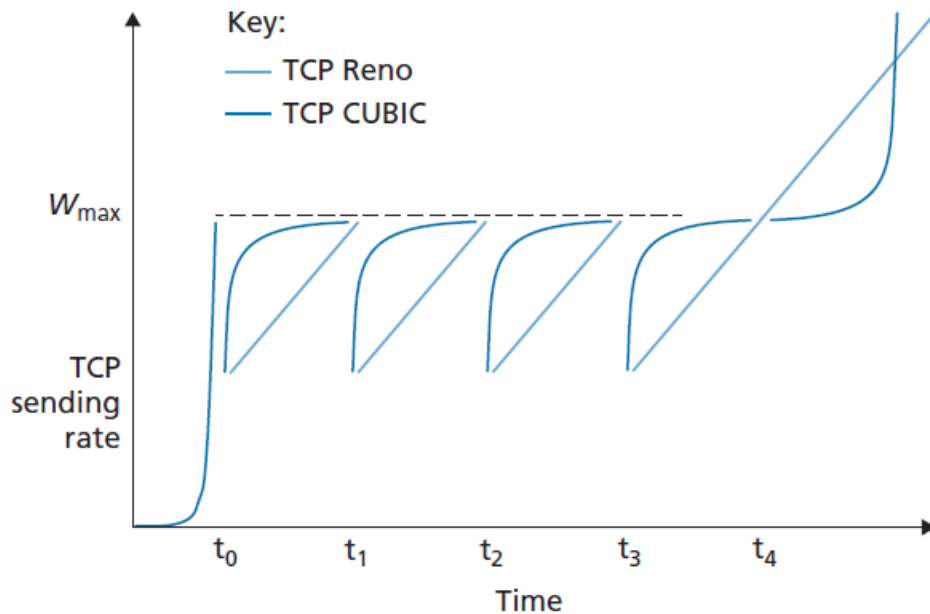
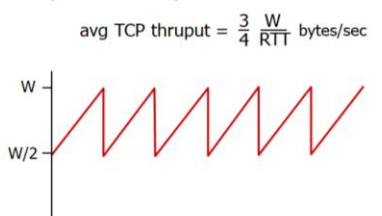


Figure 3.54 ♦ TCP congestion avoidance sending rates: TCP Reno and TCP CUBIC

- **TCP Vegas** takes a delay-based approach to proactively detect congestion onset **before** packet loss occurs.

TCP Throughput

- **W**: window size where loss occurs
 - avg. window size is $\frac{3}{4} W$



TCP Fairness

- If **K TCP sessions** share same bottleneck link of bandwidth **R**, each should have average rate of **R/K**
- It has been shown that when multiple connections share a common bottleneck, those sessions with a **smaller RTT** are able to grab the available bandwidth at that link more quickly as it becomes free and thus will enjoy **higher throughput** than those connections with larger RTTs

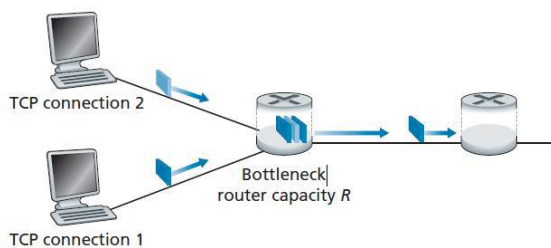
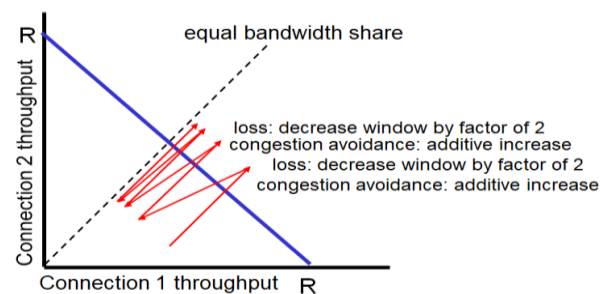


Figure 3.56 ♦ Two TCP connections sharing a single bottleneck link



Fairness and UDP

- multimedia apps often do not use TCP
 - do not want rate throttled by congestion control
- instead use UDP:
 - send audio/video at constant rate, tolerate packet loss

Fairness, parallel TCP connections

- application can open multiple parallel connections between two hosts
- web browsers do this
- e.g., link of rate R with 9 existing connections:
 - new app asks for 1 TCP, gets rate $R/10$
 - new app asks for 11 TCPs, gets $R/2$

- From the perspective of TCP, the multimedia applications running over **UDP are not being fair.**

Two types of congestion control

- End to End (Like above)
- **Network assisted**

Network assisted

- ECN (Explicit Congestion Notification)
- Routers provides **feedback** to the sender and/or receiver regarding the **congestion rate** of network
- **Two bits** in IP header marked *by network router* to indicate congestion and carried to **receiving host**
- Receiver (seeing congestion indication in IP datagram)) sets **ECE** bit on **receiver-to-sender** ACK segment to **notify sender of congestion**

